Advanced Python programming using AI tools

Lecture 13: Vibe Coding & LeetCode

Lecturer: Dr. Havana Rika

Today

1. Vibe Coding
2. LeetCode

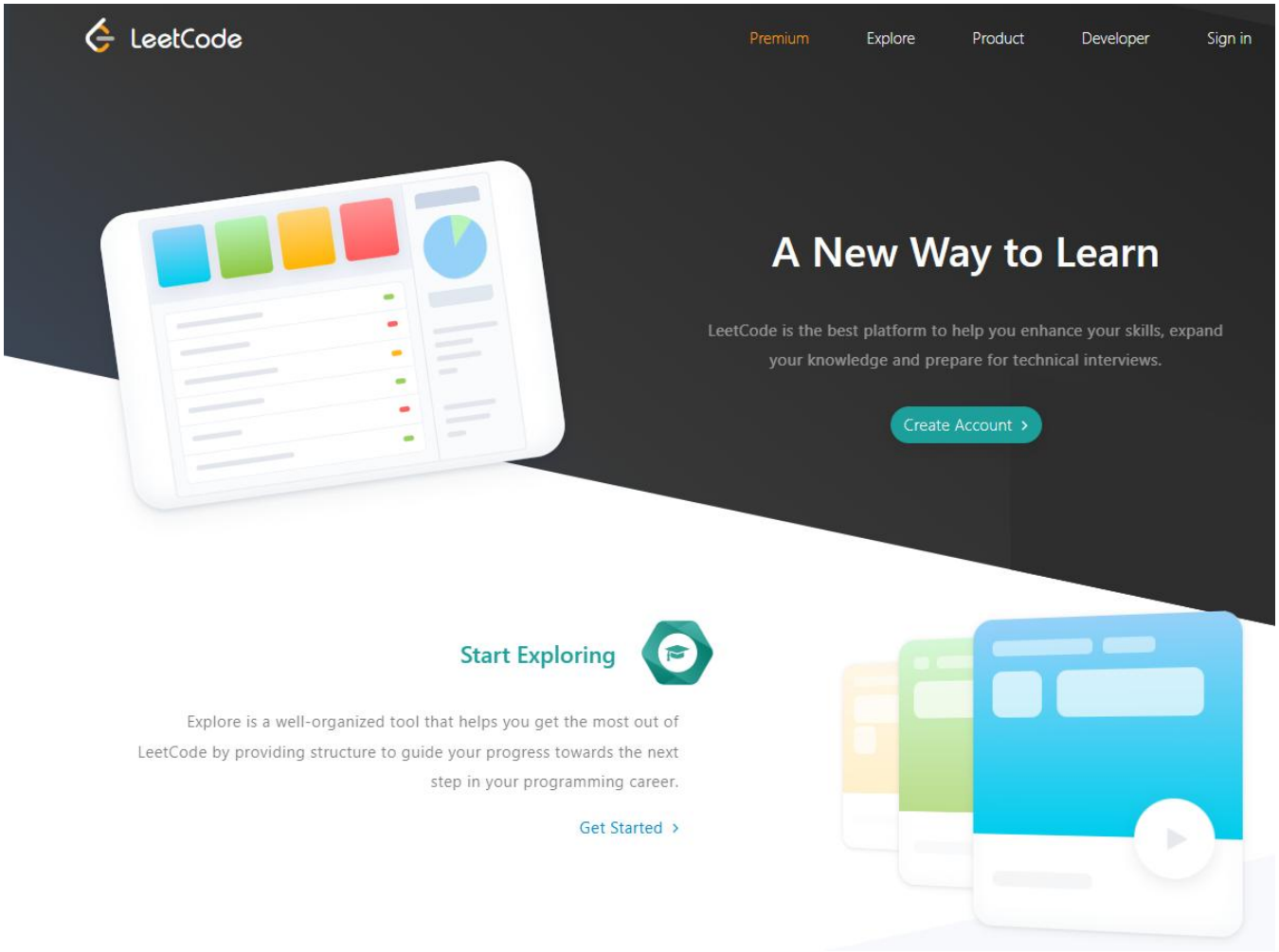
Cursor IDE

An AI-powered code editor

Google Antigravity

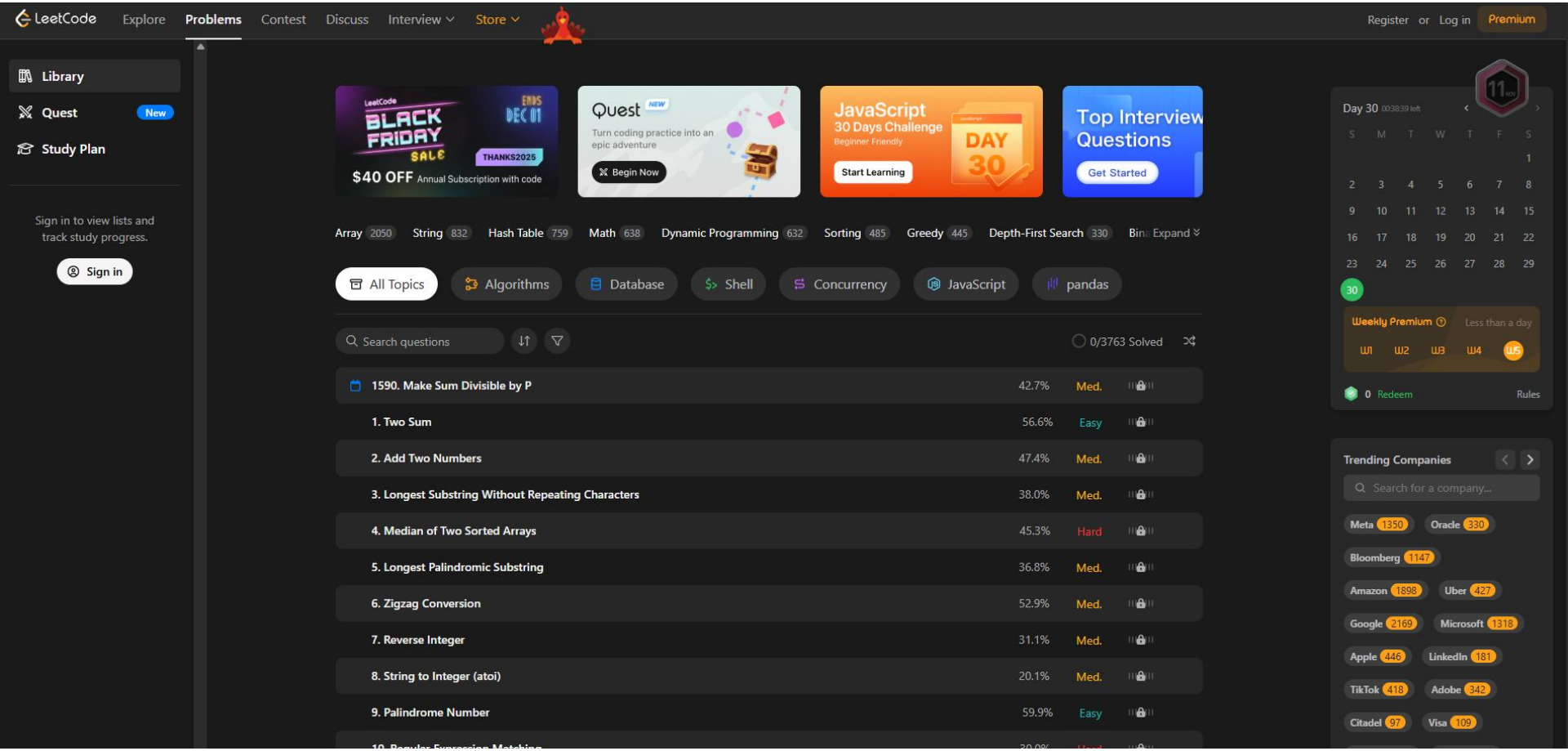
LeetCode

LeetCode



https://leetcode.com/

LeetCode Problem Set



https://leetcode.com/problemset/

THE ACADEMIC of TEL AVIV-YAFFO COLLEGE

Vibe Coding

Instead of **copying** and **pasting** code into **ChatGPT** to ask why it isn't working, to refactor it, or simply to explain it to me.

AI editors solve this problem by **integrating** GPTs **directly** into code **editors**.

By integrating directly with our code, GPTs gain **more context** about the overall project, which **significantly enhances** their **output**.

Cursor IDE

An AI-powered code editor

Google Antigravity

https://cursor.com/

What Is Cursor AI?

Cursor AI is an **AI-powered code editor** designed to make software development easier. As a fork of Visual Studio Code (**VS Code**), it retains the user-friendly interface and extensive ecosystem of VS Code, making it **easier** for **developers** already familiar with the platform to transition.

Cursor AI integrates advanced AI capabilities through OpenAI's **ChatGPT** and **Claude**. This integration allows Cursor AI to offer **intelligent code suggestions**, **automated error detection**, and dynamic **code optimization**.

https://cursor.com/

What Can Cursor AI Do?

Key autocompletion features

Cursor offers key autocompletion and predictive code features.

1. **Autocomplete and code prediction:** Cursor provides autocomplete functionality that predicts multi-line edits and adjusts based on recent changes.
2. **Code generation:** Familiar with recent changes, Cursor predicts what we want to do next and suggests code accordingly.
3. **Multi-line edits:** It can suggest edits that span multiple lines.
4. **Smart rewrites:** The editor can automatically correct and improve our code, even if we type carelessly.
5. **Cursor prediction:** It predicts the next cursor position, allowing seamless navigation through the code.

What Can Cursor AI Do?

Chat features

Cursor also integrates advanced chat features to facilitate better interaction.

1. **Codebase answers:** Query Cursor about the codebase, and it will search through the files to provide relevant answers.
2. **Code reference:** Reference specific blocks of code or files, integrating them into the context of our queries.
3. **Image support:** Drag images into the chat or use buttons to add visual context.
4. **Web search:** Get the latest information from the internet directly into code queries.
5. **Instant apply:** Implement code suggestions from the chat directly into the codebase with a click of a button.
6. **Documentation integration:** Reference popular libraries and add our own documentation for quick access.

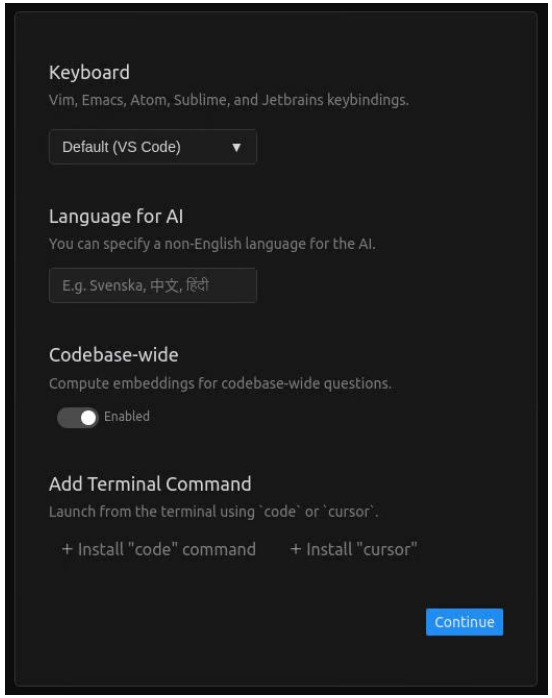
How to Install Cursor AI

•**Keyboard:** This option lets us configure the keyboard shortcuts. By default, it uses the VS Code shortcuts, which I recommend unless you are familiar with another code editor on the list.

•**Language for AI:** Here, we have the option of using a non-English language to interact with the AI.

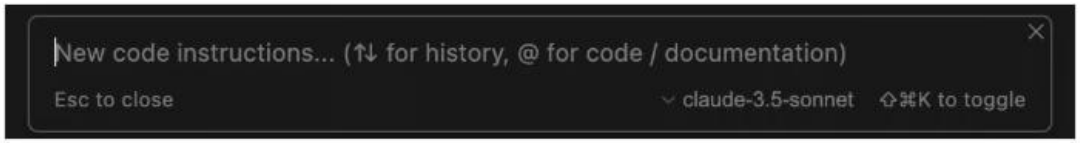
•**Codebase-wide:** Enabling this option allows the AI to understand the context of the entire codebase.

•**Add terminal command:** If installed, these allow the Cursor AI editor to run from the terminal.

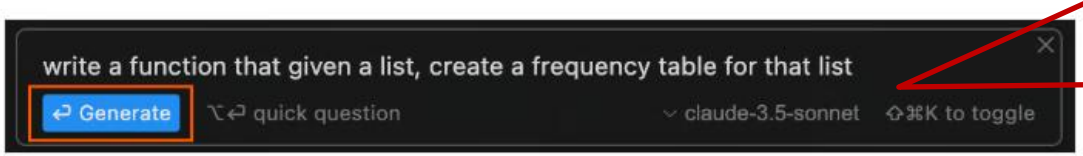


Inline code generation

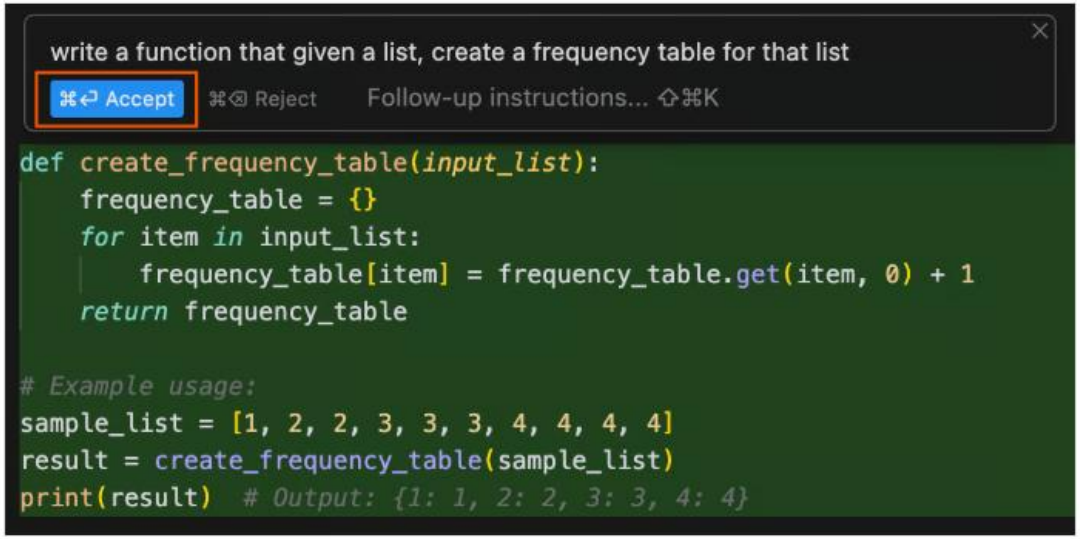
We use the Cmd+K shortcut to open the inline code generator. This opens a small prompt window where we insert a prompt to generate code:



To generate code, we type a prompt and then click the generate button:



This will generate the code, and we add it to our project by clicking the accept button:

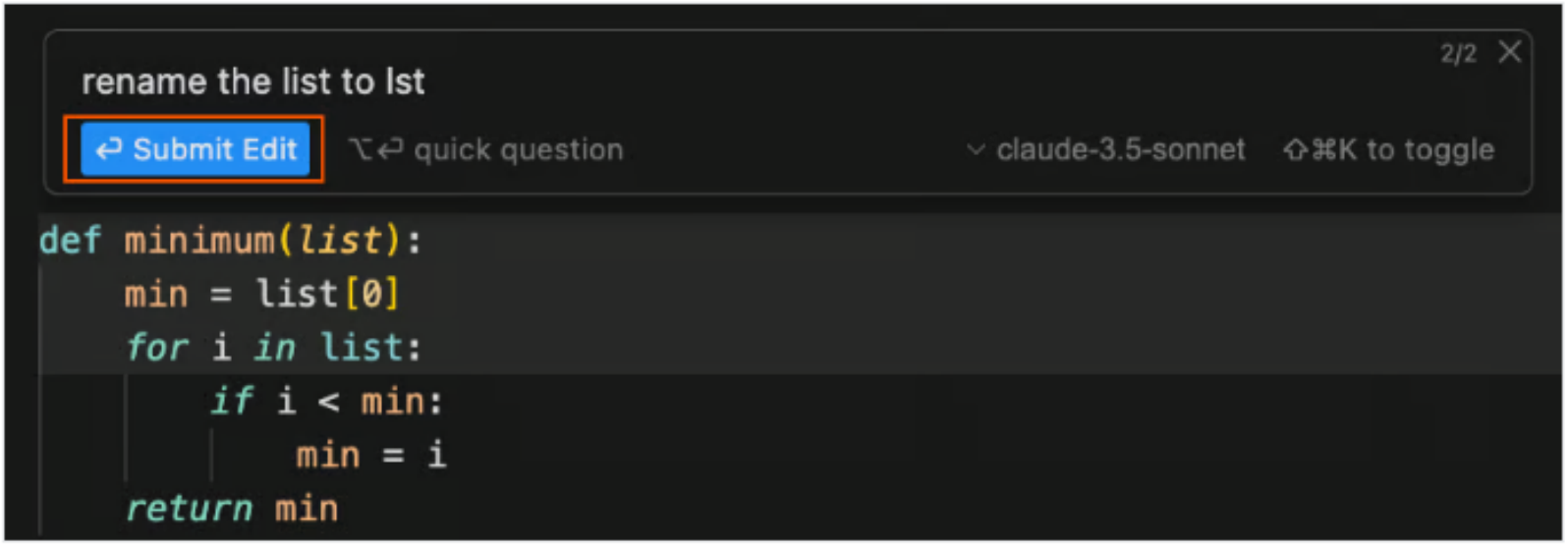


In this case, we used the cloude-3.5-sonnet model. We can select another model using the model dropdown selector

Interact with existing code

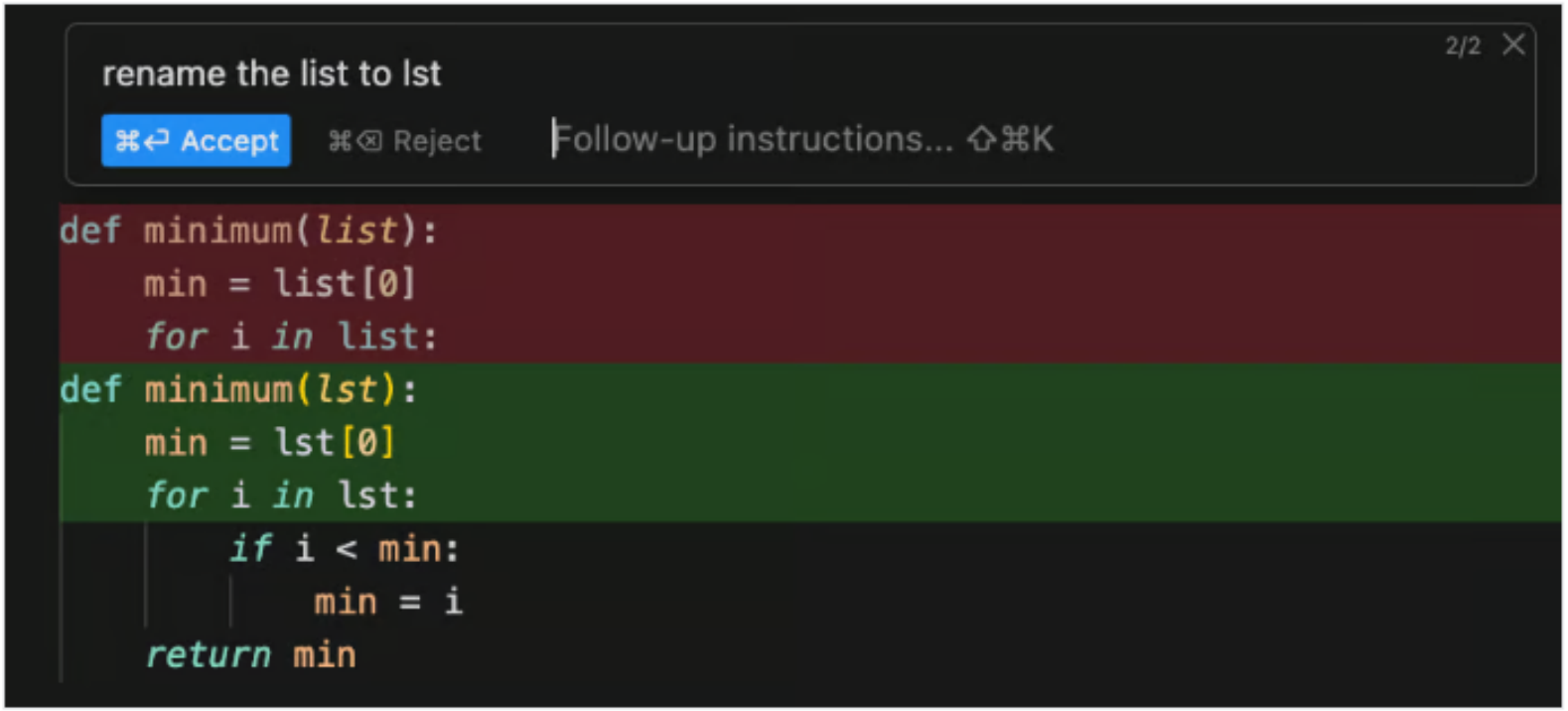
Interact with existing code

We can also use the inline chat to interact with existing code by selecting the relevant code before using the `Cmd+K` shortcut. This can be used to make changes to the code, such as refactoring, or to ask questions about the code. After typing the prompt, we click on the *Submit Edit* button to get the modifications:



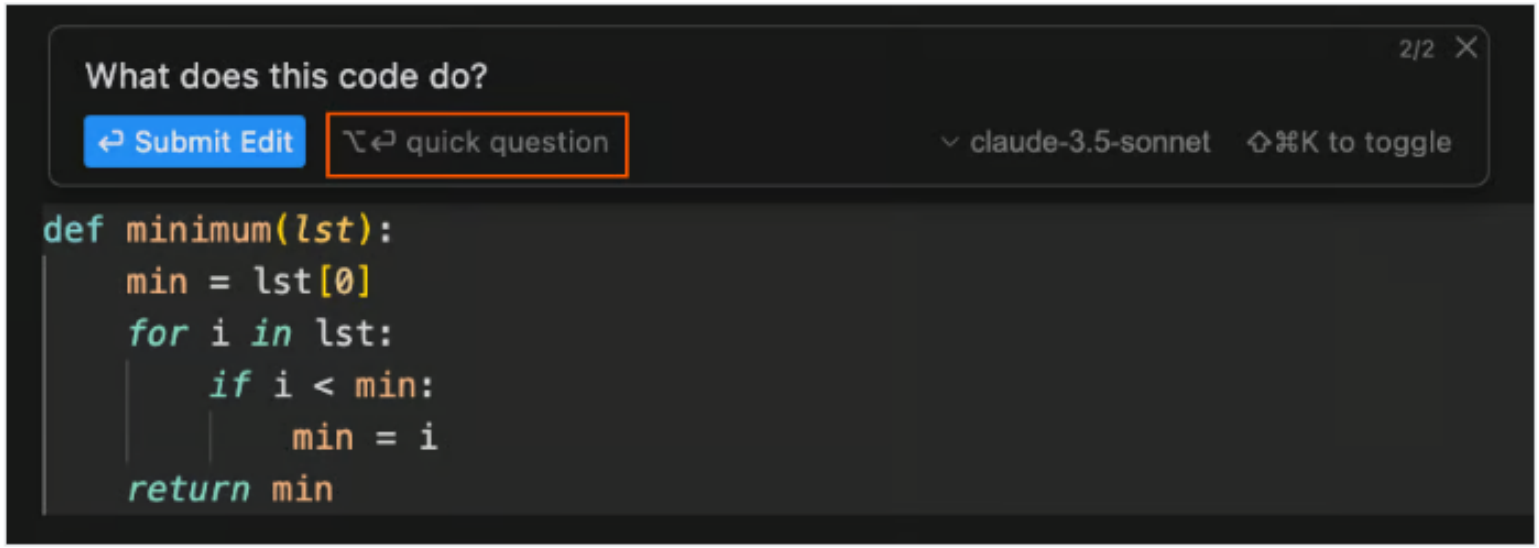
Interact with existing code

Code changes in the Cursor are presented as a diff. The red lines represent lines that will be deleted by the change, while the green ones represent the new changes that will be added:



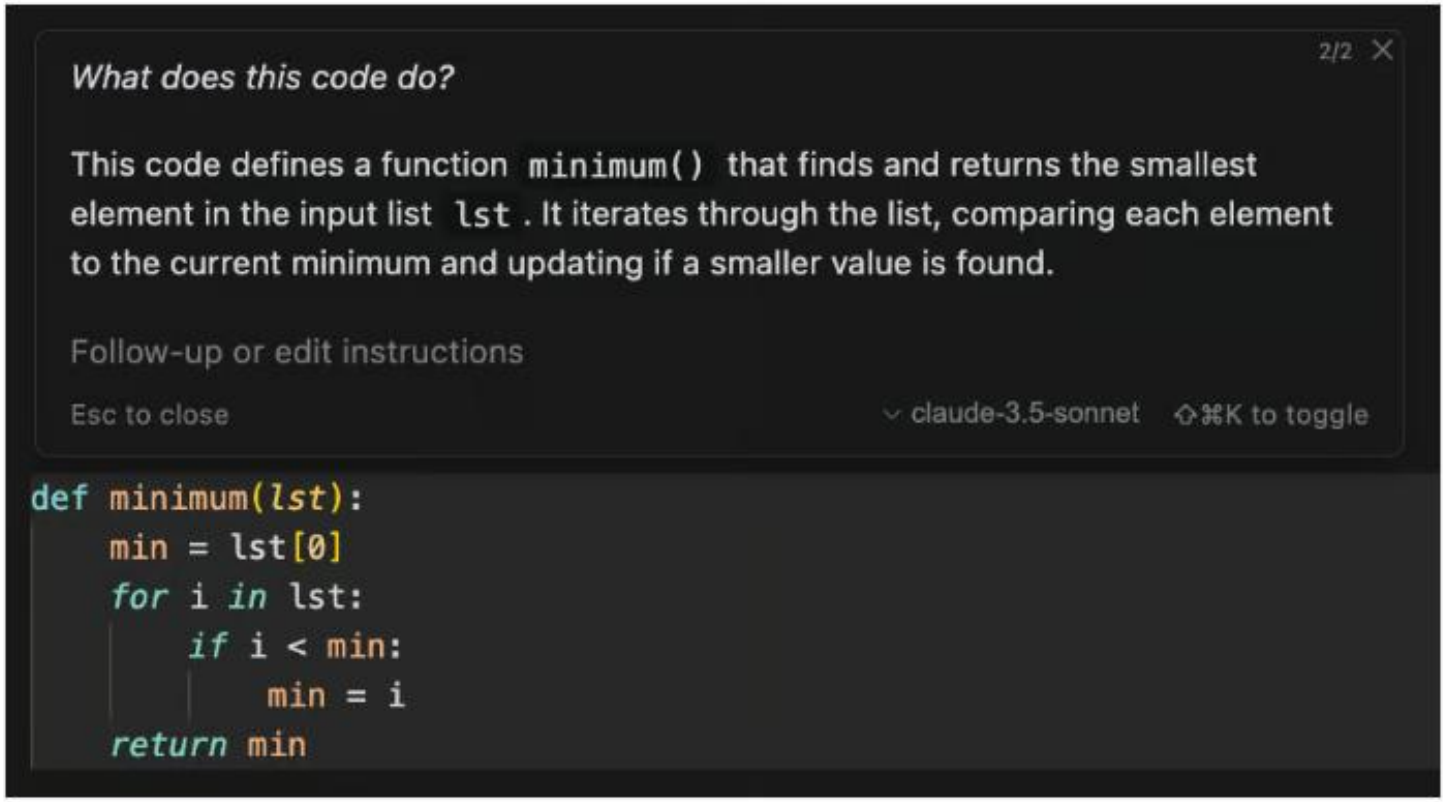
Asking questions on existing code

In the same way, we can ask questions about a piece of code by selecting it and using the `Cmd+K` shortcut. In the case of a question, we click the `quick question` button to submit the prompt:



Asking questions on existing code

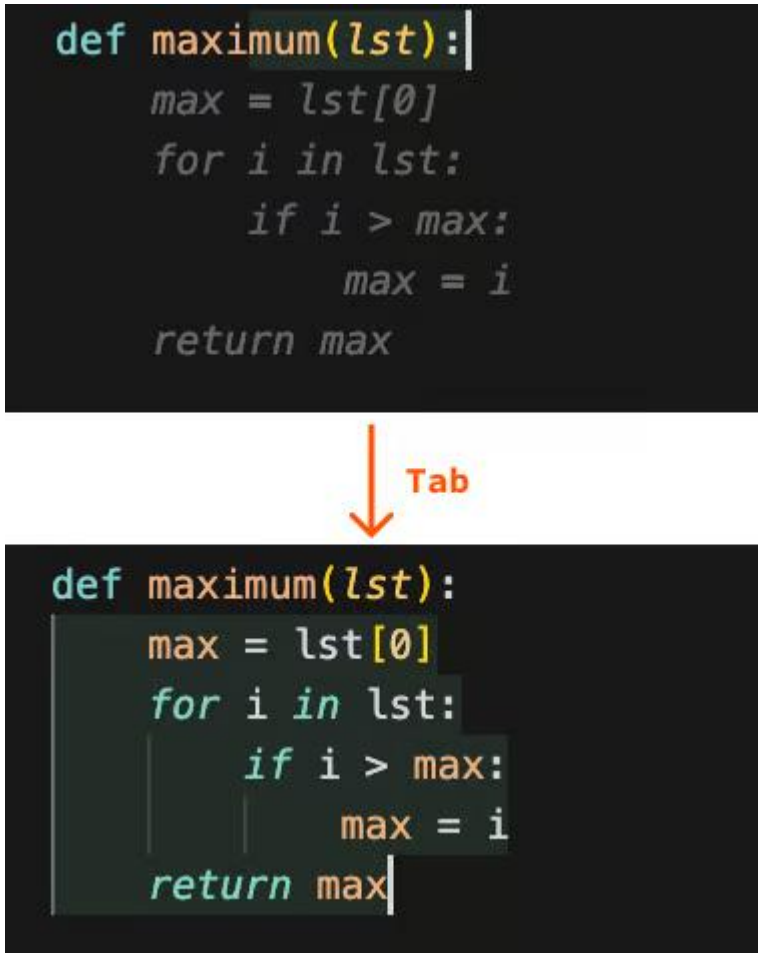
After submitting the question, the system will generate the answer and display it in the following manner:



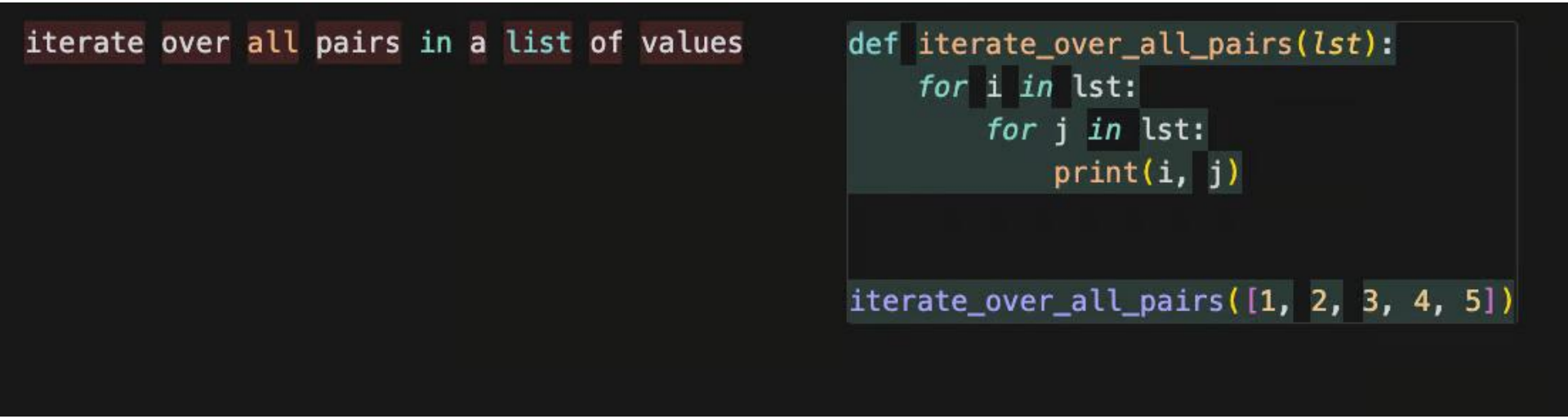
Autocompletion with tab

While writing code, Cursor will suggest code completions generated using AI. Similar to traditional code completion, we can use the Tab key to incorporate these suggestions into our code.

For example, let's say we start implementing a function named maximum(). Cursor will recognize our intent and suggest an appropriate implementation. By pressing Tab, we can add the suggested code:



Autocompletion with tab



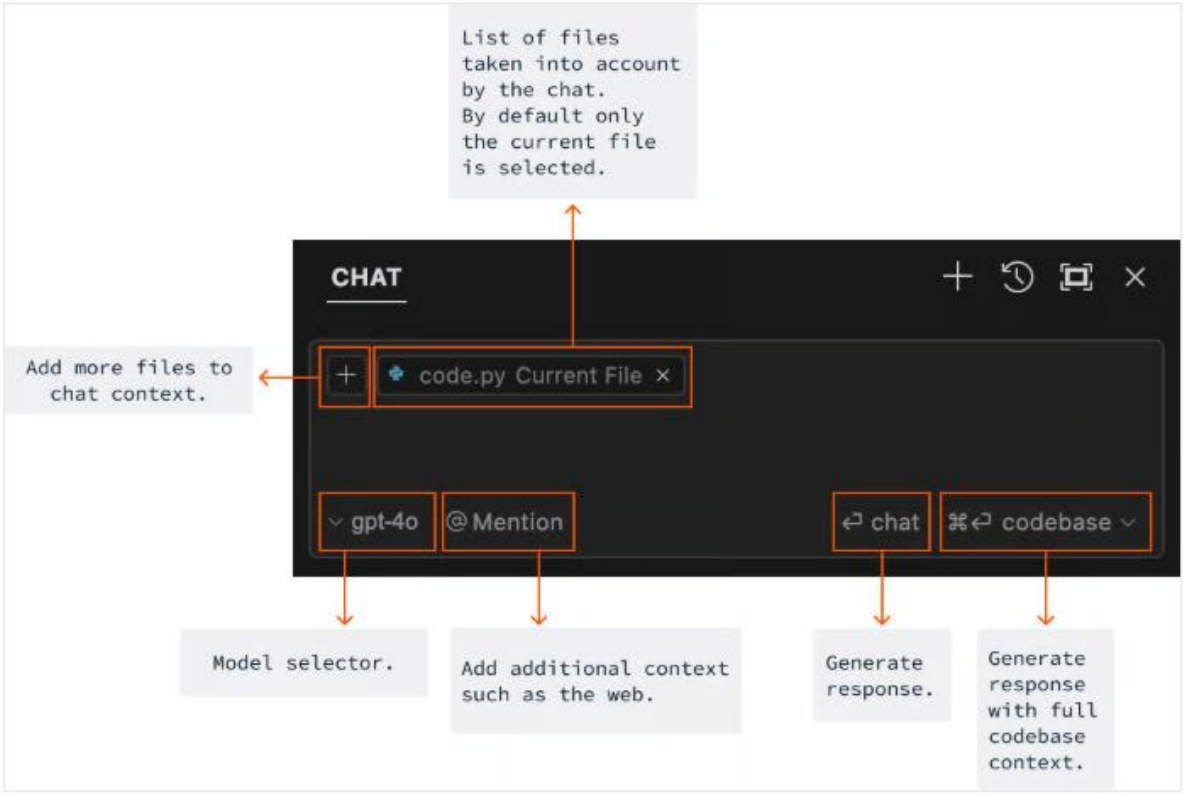
Autocomplete also works with code written in natural language.

For example, if we want to create a double for loop to iterate over all pairs in a list, we can simply describe this in plain text.

Cursor will then provide the corresponding autocomplete suggestion, which can be integrated by pressing the Tab key.

Chat interface overview

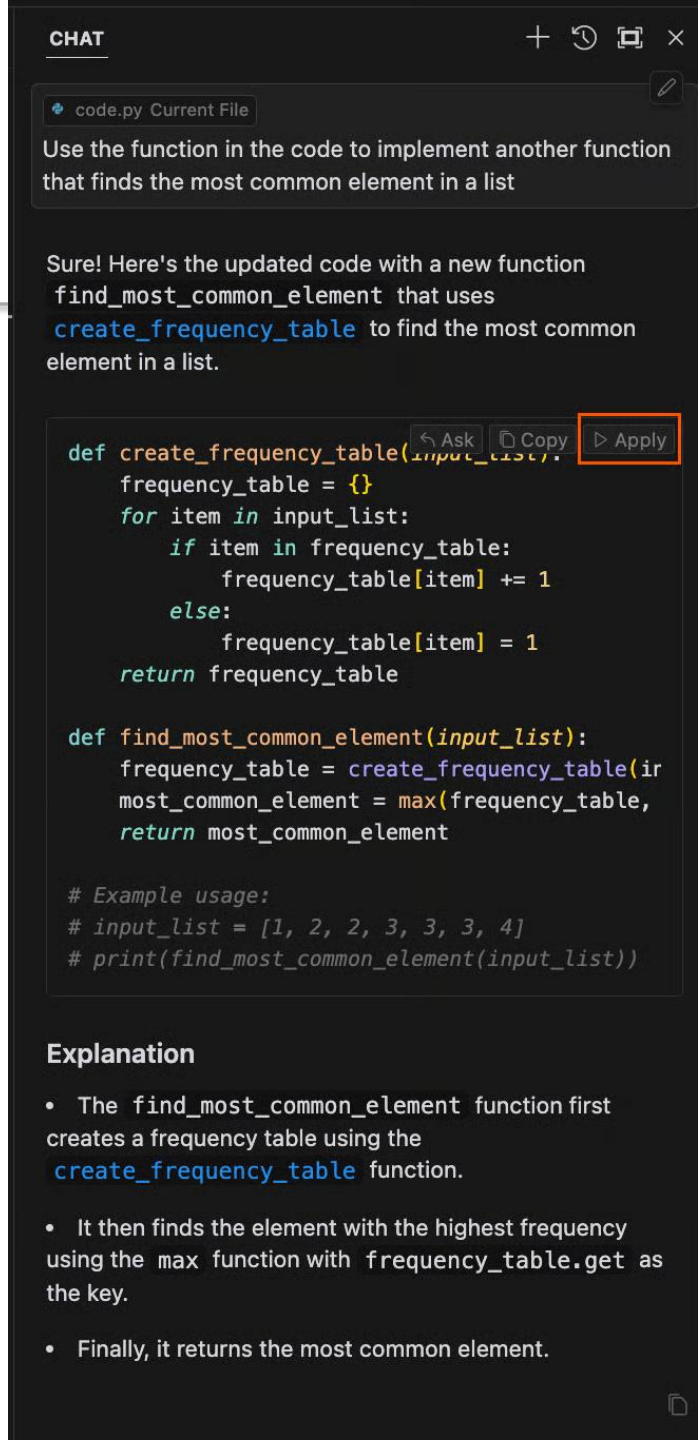
To open the chat window, use the `Cmd+L` shortcut. The chat window is more versatile than the inline generator, as it allows us to not only generate code but also ask questions. Here's an overview of the chat interface:



Code generation with chat

Similarly to generating with the inline chat, we can also **generate code** using the **chat feature**.

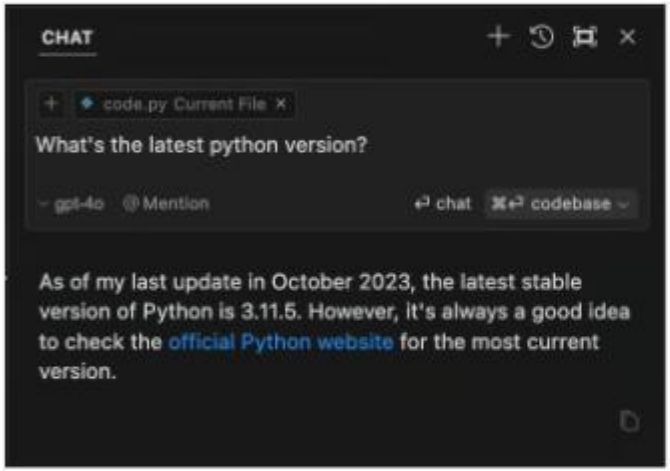
The **code** generated in the chat can be **integrated** into the **project** by clicking the ***Apply*** button at the top right corner of the code window.

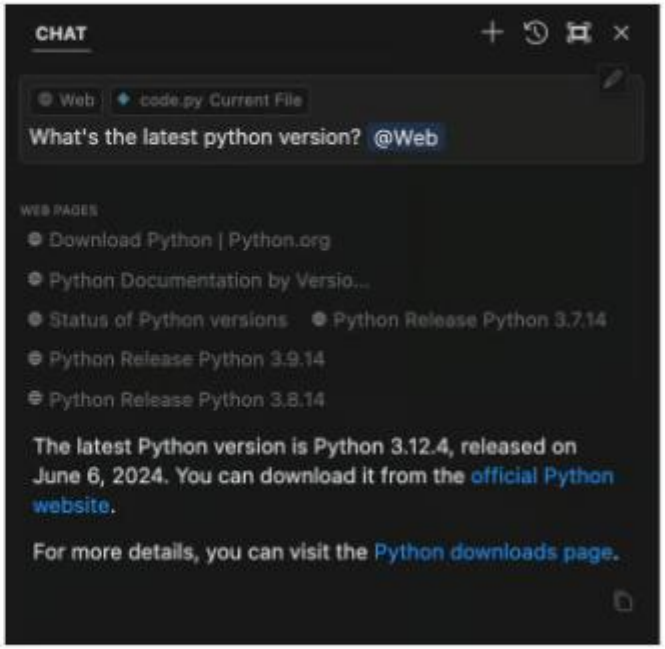


Enhancing query context with @

Perhaps the most important feature of the chat window is the `@ Mention` option. This option enables us to provide more data for the AI to generate the response. This ranges from simple files and folders to web searches or giving the AI access to a GitHub repository.

For example, we can use `@Web` to allow the AI to search the web for the answer.

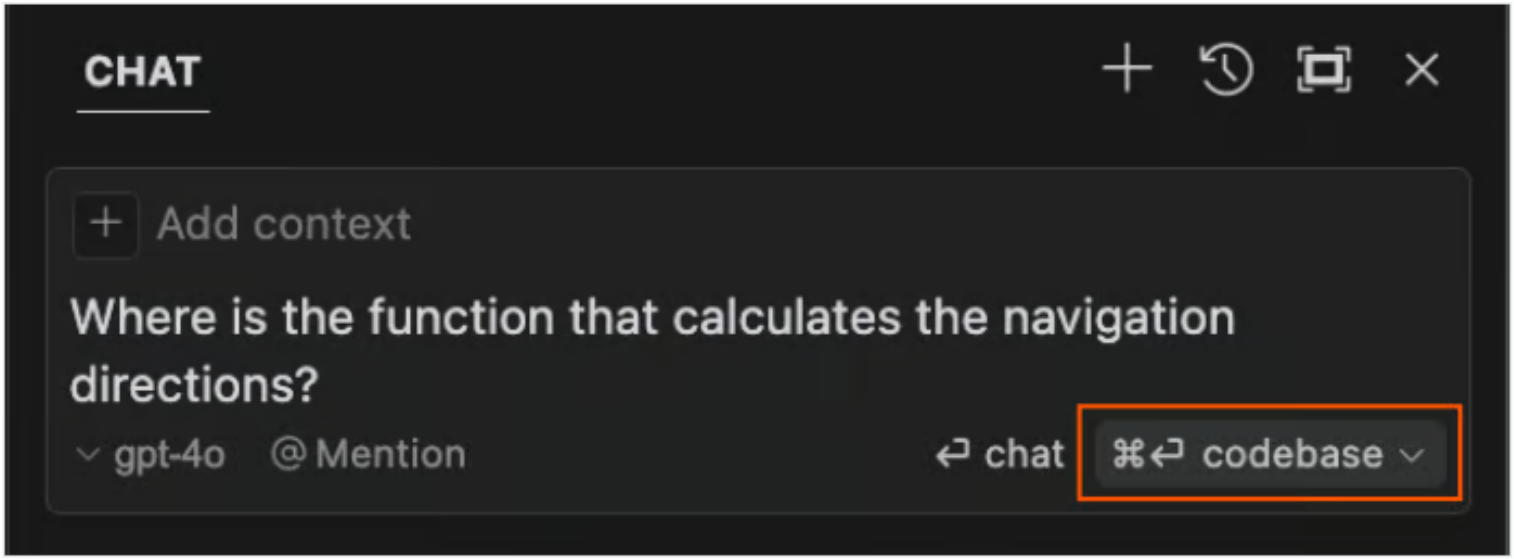




Keep in mind that in some cases it might be problematic to share the whole code base or a private GitHub repository with the AI. We should be mindful of what we share with the AI and avoid sharing sensitive or private data.

Global code base questions

One of the features that I find the most useful when working on bigger projects is the ability to quickly find a piece of code by asking a question with the full codebase as scope. Recently, I wanted to locate a function in a project that calculates the navigation direction in an app. With Cursor, I could very simply locate it by describing what the function does:

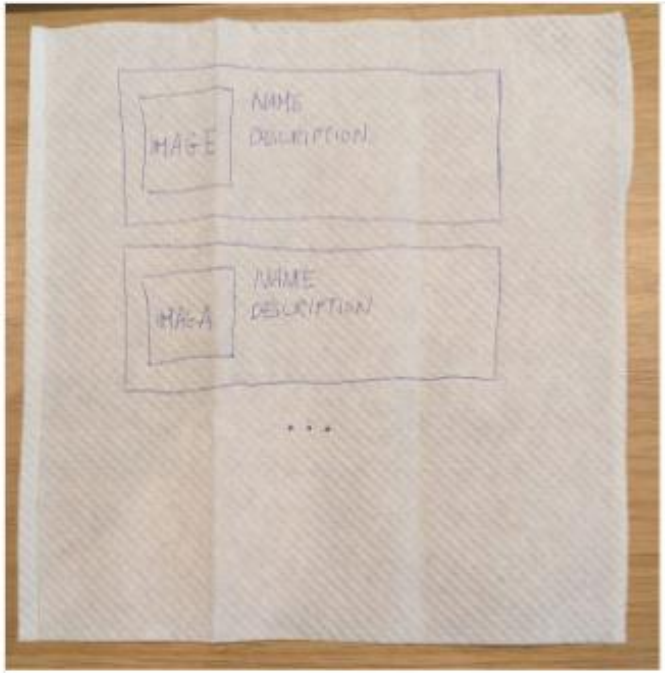


Global code base questions

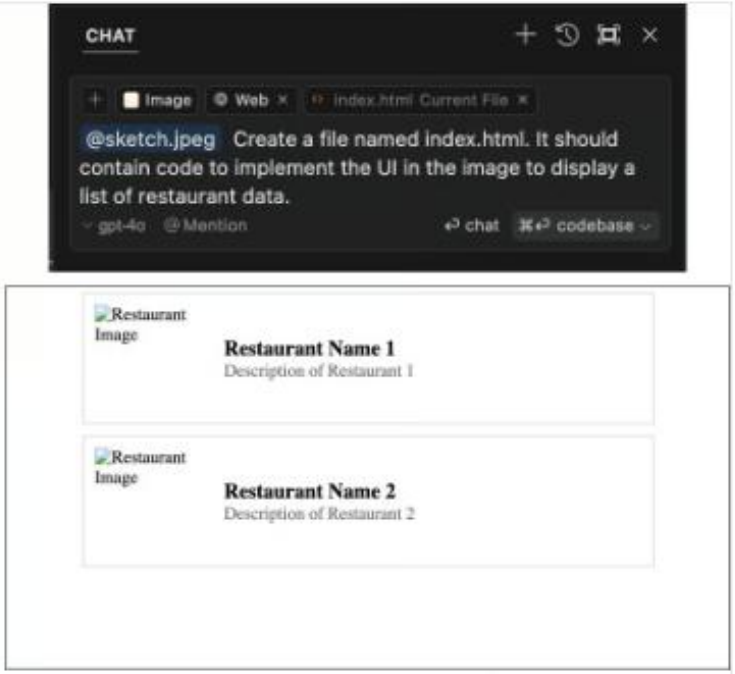
Note that we use the `codebase` option in this case. Although Cursor didn't display the actual code for some reason, clicking the code box still opened the correct file and scrolled to the function I was looking for:

Image support

The Cursor chat also supports image inputs. For example, we could sketch a UI design for a website and ask it to generate the HTML and CSS code for it. To add an image, we can drag and drop it into the chat window.



UI napkin sketch

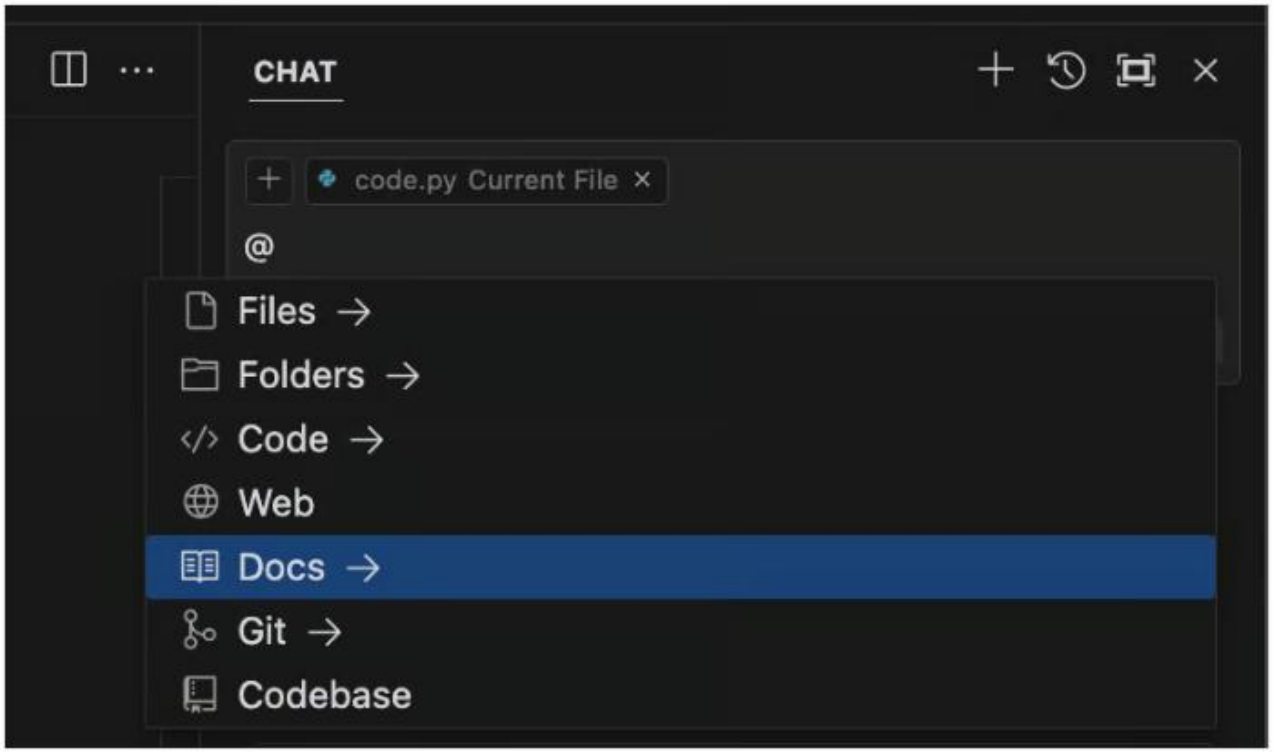


Chat input and rendered HTML code output

Adding documentation

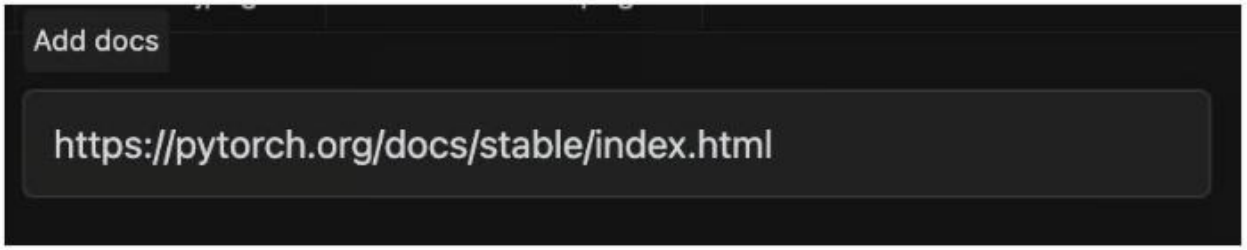
A very useful feature of Cursor AI is the ability to add documentation references. This is especially useful for lesser-known or private libraries whose documentation might not have been used in the AI training process.

To add a documentation entry, we use the `@` symbol and then select `Docs` from the dropdown menu:

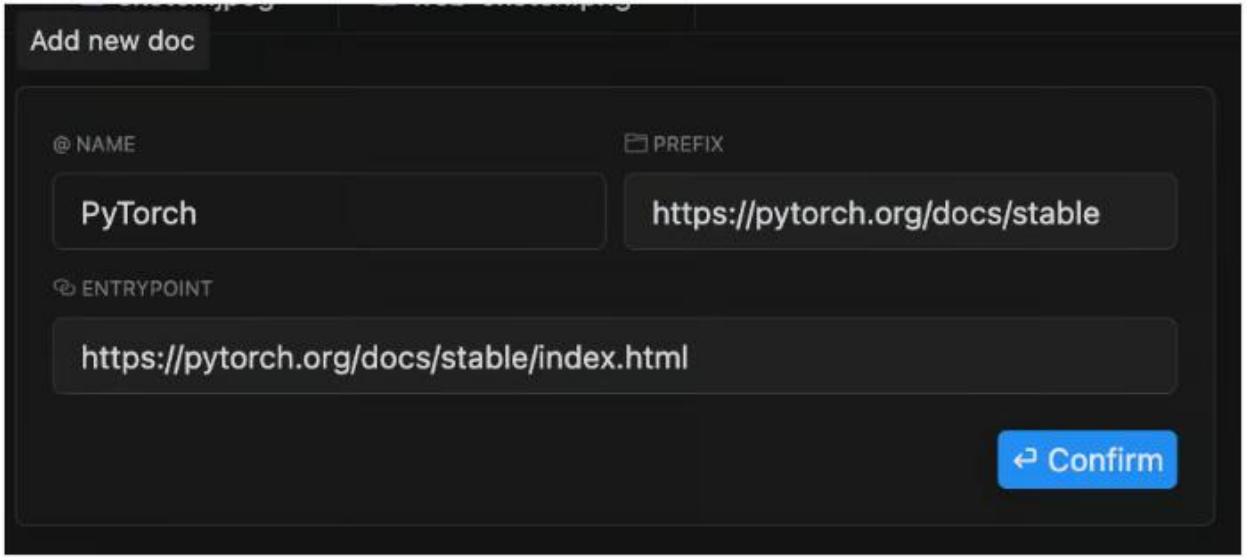


Adding documentation

This will open a window requesting a URL for the documentation. Let's add the PyTorch documentation as an example:

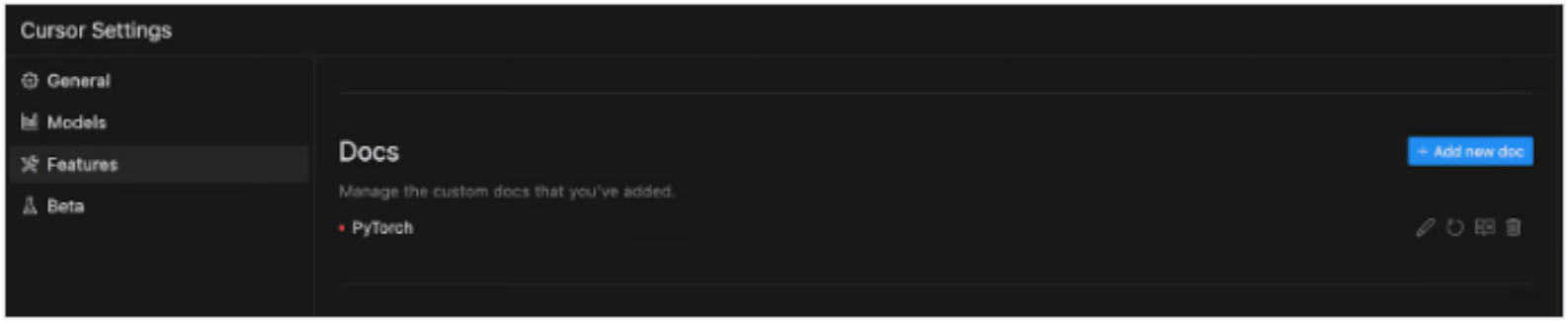


After inserting the URL, we can give the documentation entry a name. In this case, we use `PyTorch`. We can then use this name to refer to this documentation in the chat prompt using `@PyTorch`.



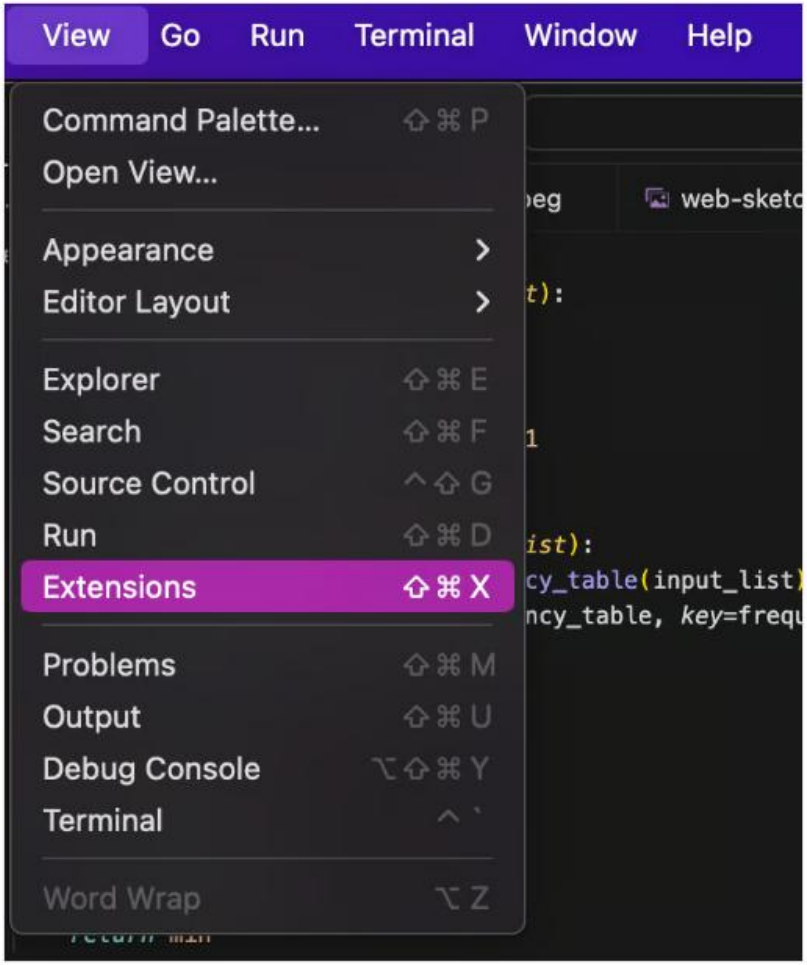
Adding documentation

Documentation references can also be managed in the *Features* tab from the Cursor settings:



Additional Features and Benefits

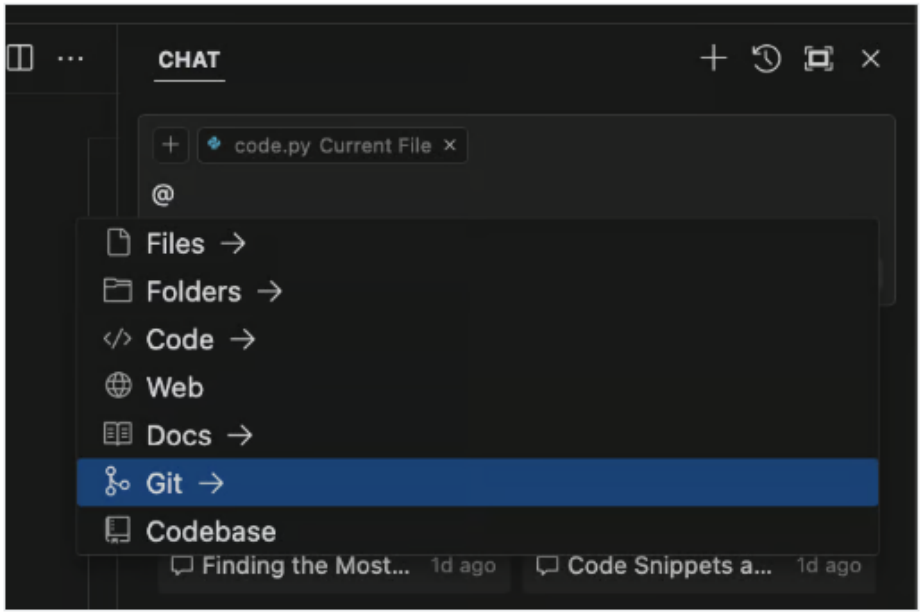
Because Cursor is built on top of VS Code, it inherits from its rich extension ecosystem. We can access these in the `View` menu.



Collaboration with others

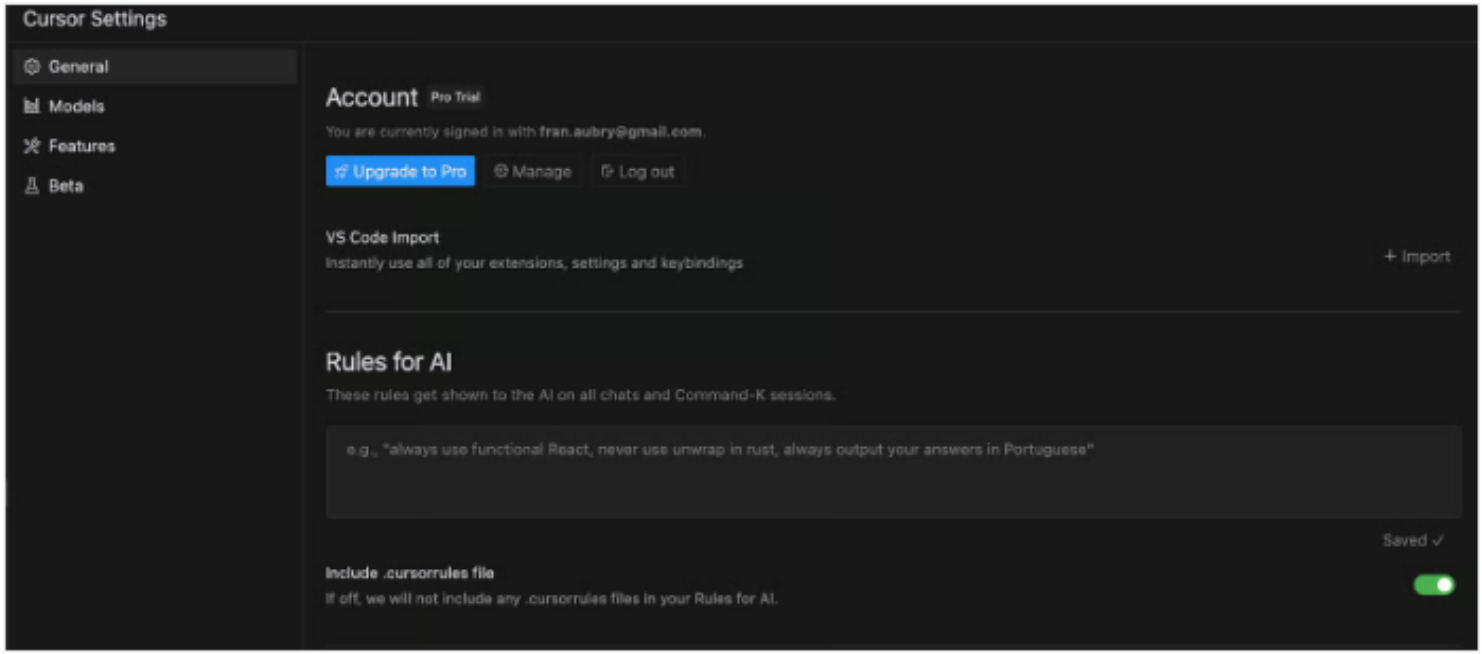
Using collaborative tools such as Git with Cursor is similar to using them with any code editor. These tools are not dependent on how the code was written. There are extensions specifically designed to assist with Git.

Remember that Cursor's chat allows you to use Git repositories within context using the `@` operator. Be mindful that this should be used with caution if the repository contains private data.



Setting custom AI rules

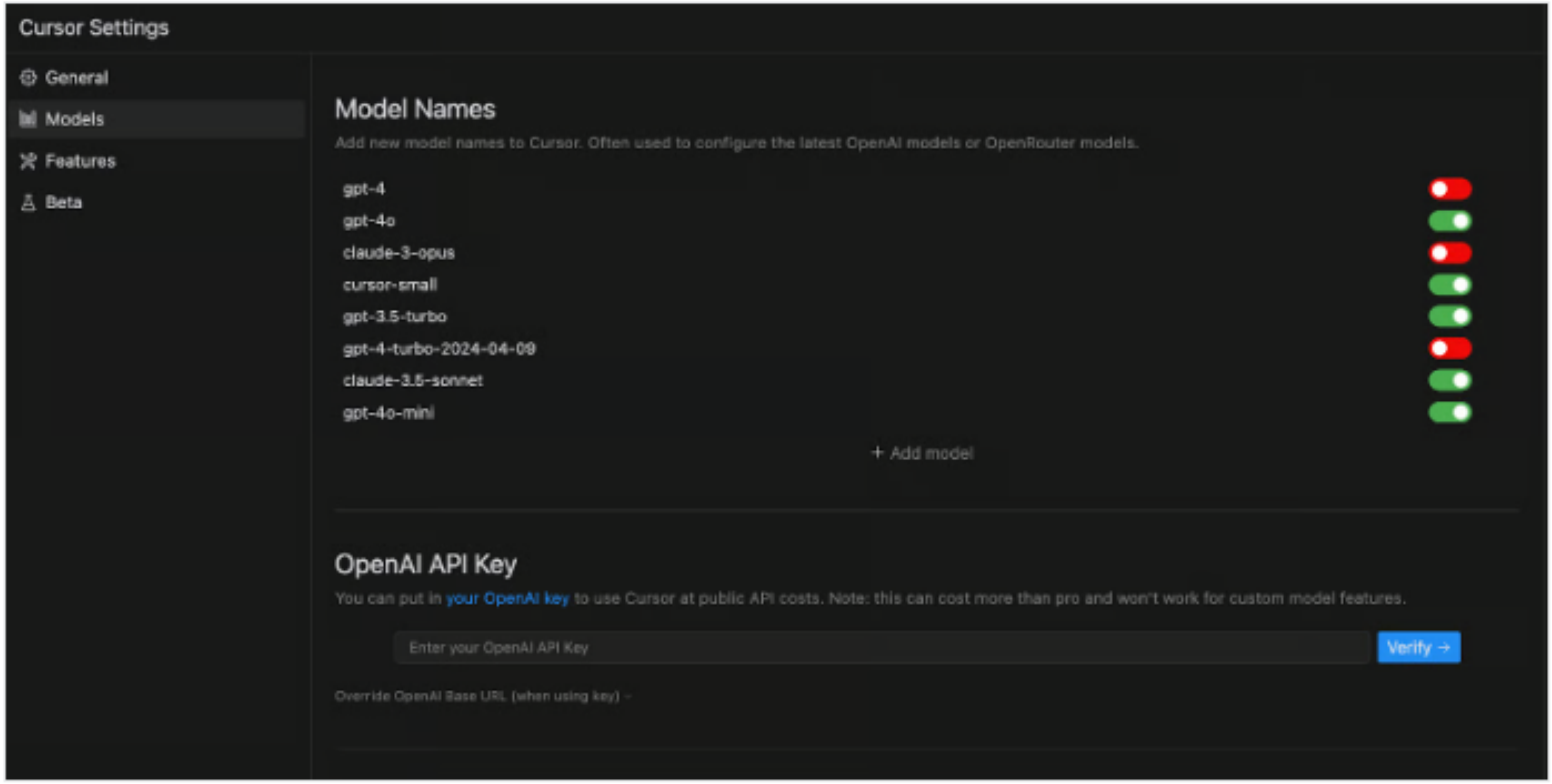
Cursor allows us to guide the AI using specific rules. These are accessible under the general settings menu:



These rules can modify the AI's behavior without needing to prompt it repeatedly. For example, we can ensure the AI always uses type hints in Python by adding a rule such as "Always use type hints in Python function definitions."

Custom AI models

Another interesting feature of Cursor is the ability to add other AI models. This option can be found under the `Models` settings:



Cursor AI vs GitHub Copilot

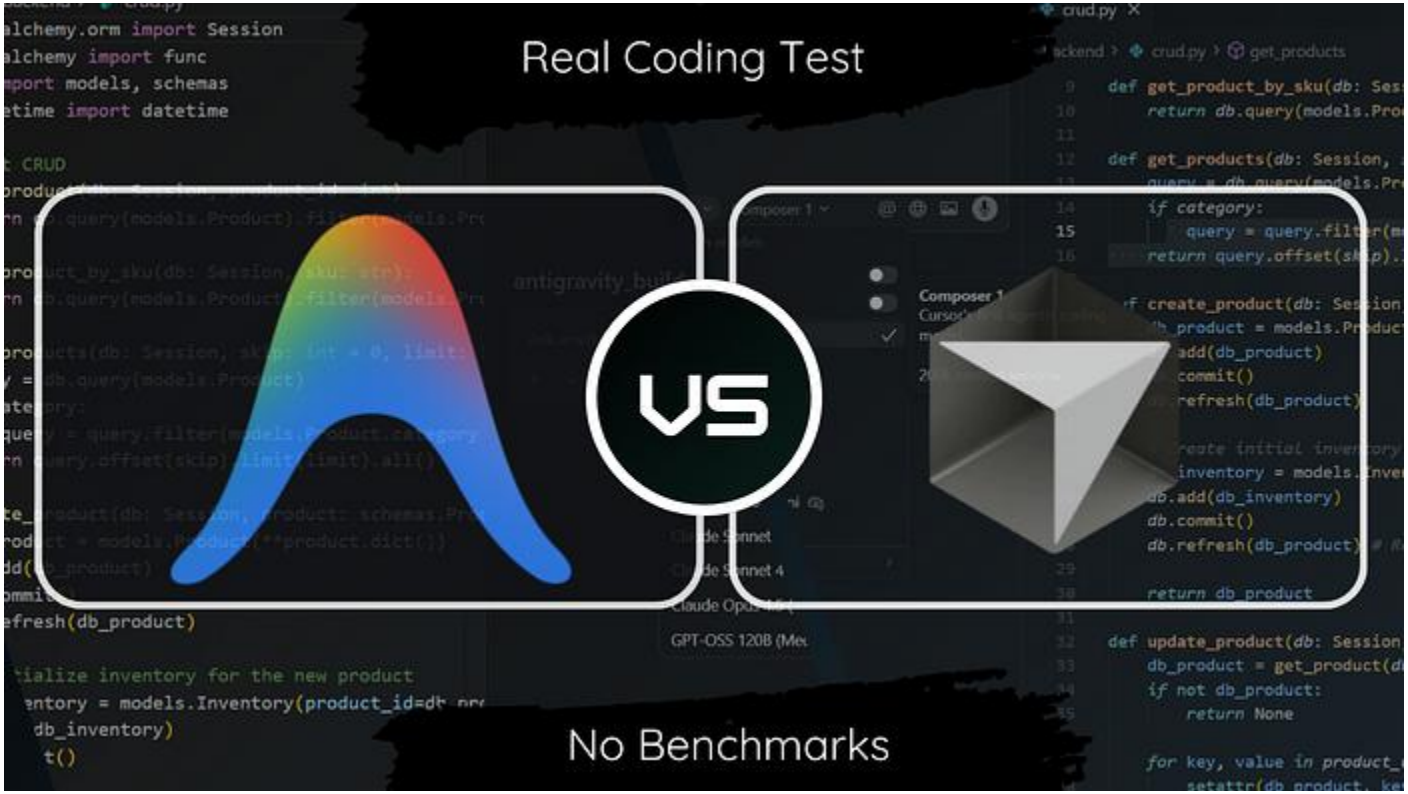
Both tools provide **real-time code suggestions** and support for multiple languages and frameworks.

Cursor AI may be advantageous for specialized tasks due to its **deep integration**, while **GitHub Copilot's extensive IDE** support and straightforward setup make it accessible to a broader audience.

Ultimately, the choice between Cursor AI and GitHub Copilot may depend on factors like customization **needs**, integration **preferences**, and **budget**.

Both tools aim to **enhance coding** efficiency in different ways.

Cursor vs Antigravity



https://youtu.be/saPRIExh5w4

Cursor vs Antigravity

Speed Winner: Cursor was significantly **faster**. It began generating code and installing dependencies while **Antigravity** was still in the "planning" and task-list phase.

Workflow Styles: Cursor uses a seamless "Agent Mode" that plans and executes tasks simultaneously. **Antigravity** uses a structured "Planning Mode" where the user must approve a multi-step plan before coding begins.

Automated Testing: Both tools can open a browser to test their own code. **Cursor** provides more detailed, step-by-step logs of these tests, while **Antigravity** uses an external automated Chrome instance.

The Result: Despite the speed difference, **both** tools produced nearly identical, **high-quality**, and functional applications.

Final Verdict: Cursor is better for raw speed and a fluid experience, while **Antigravity** offers a more rigid, step-by-step planning structure.